

Mário

Etapa 1

Temos que pensar na lógica do nosso jogo!

Para isso, precisamos coletar informações quanto ao planejamento. Qual a primeira ação que vocês identificam quando olham para o jogo?

1. O Mário Pula
2. O jogo mexe
3. Avisa o gameOver da partida
4. Reinicia o jogo se o usuário quiser

A partir dessa lógica, vamos construir nosso jogo!

Estilizando os elementos

Para fazermos o Mário pular, antes de tudo, precisamos que o Mário e o obstáculo existam no nosso HTML

Dessa forma, é preciso criar a tela, o cenário e os nossos personagens no HTML! E você consegue pensar em quais tags HTML vamos utilizar?

OBS: peguem as imagens no repositório indicado pela professora!

```
<div class="jogo">
  <div class = "cenario">
    
    
  </div>
</div>
```

| E você sabe porque precisa ter duas divs?

Como precisamos ter a tela fixa (para representar a tela do computador) e o cenário em movimento, precisamos que haja essas tags HTML diferentes

para que a animação (que ainda vamos colocar) seja aplicada somente ao cenário!

Dessa forma, vamos adicionar o código:

```
.jogo {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  width: 100%;  
  height: 100%;  
}  
  
.cenario {  
  width: 1080px;  
  height: 380px;  
  border: 2px solid black;  
  background-image: url("cenario.jpg");  
}
```

E aí? Como que ficou o código?

Agora, precisamos falar para os elementos dentro do meu cenário até onde que eles podem se movimentar (nesse caso, a tela do meu jogo!)

Dessa forma, utilizamos o position "relative" no pai e, nos filhos, aplicamos o position "absolute" para que eles se movimentem sabendo o limite do cenário!

Além disso, podemos movimentar os elementos das mais diversas formas no cenário , utilizando left, right, bottom e top!

Estilização do cenário:

```
.cenario {  
  width: 1080px;  
  height: 380px;  
  position: relative;  
  border: 2px solid black;  
  background-image: url("cenario.jpg");  
}
```

Estilização dos personagens:

```
.mario {  
  position: absolute;  
  height: 200px;  
  width: 120px;  
  bottom: 0px;  
  left: 100px;  
}
```

```
.obstaculo {  
  height: 160px;  
  width: 100px;  
  position: absolute;  
  bottom: 0px;  
}
```

O "bottom" define o quão afastado o elemento vai está do chão. Enquanto o "left" define o quão afastado o elemento vai está da esquerda!

Estranho? Coloque no "left", o valor que você acha que o Mário tem que está (lembre-se: ele vai se afastar da borda esquerda do jogo!)

Fazer o cenário se movimentar

Para isso, vamos aplicar prioridades de animação ao nosso cenário!

▮ A propriedade "keyframes" permite a criação de animações intermediárias.

Caso fique curioso, teste o código abaixo e veja o que acontece!

```
.quadrado {  
  width: 50px;  
  height: 30px;  
  animation: quadrado linear 1s infinite;  
  background-color: blue;  
  position: relative;  
}  
  
@keyframes quadrado {  
  0% {  
    left: 100px;  
    background-color: red;  
  }  
  50% {  
    border: 2px solid black;  
    bottom: 50px;  
    background-color: yellow;  
  }  
  100% {  
    background-color: green;  
  }  
}
```

Vamos fazer o mesmo com o cenário!

A única diferença é que ele vai se movimentar para uma direção só: **esquerda ou direita**. O importante é que haja o deslocamento horizontal!

É o mesmo que dizer:

“Olha, no início da animação, posicione o elemento dentro de cenário, depois, empurre ele bem para fora. E não esqueça de repetir infinitamente!”

```
.cenario {
  width: 1080px;
  height: 380px;
  overflow: hidden;
  position: absolute;
  border: 2px solid black;
  background-image: url("cenario.jpg");
  background-repeat: repeat-x;
  animation: moverCenario 2s linear infinite;
}

@keyframes moverCenario {
  0% {
    background-position: 0px;
  }
  100% {
    background-position: -2000px;
  }
}
```

Fazer o Mário pular

Aqui também utilizamos animação, só muda a direção que vamos trabalhar!

Afinal de contas, quando o Mário pula, ele vai para onde? (Topo e depois volta para o chão)

Vamos primeiro definir o valor máximo que meu Mário vai pular, ou seja, o topo e o valor para ele voltar ao chão! Depois, disso, vamos deixar a animação mais suave!

Quando o Mário pula, eu sei que ele começa no bottom = 0, quanto ele estiver no topo posso falar que bottom será = 180 e, depois, ele volta para o chão → bottom = 0.

Fique a vontade para alterar a qual altura o Mário vai pular no "bottom"

```
@keyframes jump {  
  from {  
    bottom: 0px;  
  }  
  to {  
    bottom: 180px;  
  }  
}
```

- Vamos vê se deu certo? Tudo o que precisamos fazer é adicionar a animação que acabamos de criar, no elemento que a gente quer que pule!

E aí, vamos testar?!

```
.mario {  
  position: absolute;  
  height: 200px;  
  width: 120px;  
  bottom: 0px;  
  left: 100px;  
  animation: jump 0.5s linear infinite;  
}
```

Dessa forma, nossa animação terá uma duração de 0.5 segundos, com a mesma velocidade e que vai se repetir infinitamente!

Fazer o obstáculo se movimentar

Vamos utilizar a mesma lógica do Mário para adicionar o efeito de movimentação no nosso obstáculo!

Tudo o que precisamos fazer é dizer para ele: preciso que você ultrapasse a caixa do meu jogo infinitamente!

Então, vamos trabalhar com qual direção? (Podemos pensar em qualquer uma que esteja dentro do deslocamento horizontal!)

- Com a esquerda (left)!

E quantos segundos que essa animação vai durar?

- Vamos testar com 5 segundos!

Ela vai ser infinita?

- Com certeza!

A partir dessas respostas, vamos criar nossa animação e vê se ela vai dá certo!

```
@keyframes movimentacao {  
  0% {  
    left: -400px;  
  }  
  50% {  
    left: 900px;  
  }  
  100% {  
    left: 2000px;  
  }  
}
```

```
.obstaculo {  
  height: 100px;  
  width: 100px;  
  position: absolute;  
  animation: movimentacao 2s linear infinite;
```

```
    bottom: 0px;  
}
```

“Quando você tiver em determinado segundo, eu preciso que você tenha andado esse valor para a esquerda.”

Por fim, adicionar na nossa div o overflow, para que o pai esconda tudo o que ultrapassa a largura dele!

```
.cenario {  
    width: 1000px;  
    height: 380px;  
    overflow: hidden;  
    position: absolute;  
    border: 2px solid black;  
}
```

Vamos testar para vê se deu certo!

Avisa o gameOver da partida

Pronto! Agora, vamos fazer a mágica acontecer.

Afinal de contas, eu preciso saber quando meu Mário vai pular (não quero que ele fique igual pipoca na panela infinitamente!) e quando que ele vai perder o jogo: ou seja, encostar no obstáculo!

1. Primeiro, precisamos saber se o Mário encostou

Para o Mário encostar no obstáculo, podemos pensar o contrário: quando o obstáculo encostar no Mário! Já que ele é o objeto fixo do meu jogo.

Então quando ele estiver afastado da esquerda o mesmo valor que o Mário está, ou melhor, estiver próximo desse valor (por exemplo: 99.8), eu sei que tem chances do Mário ter encostado!

Como estamos trabalhando com animação, não é muito recomendado utilizar um valor absoluto. Pois os números se alteram muito rápido.

Como sabemos que o left do Mário é 100, precisamos validar se o left do obstáculo é menor que 100.

Caso tenha alterado utilize o valor do seu código!

Para isso, vamos seguir as etapas abaixo:

- **Selecionar os elementos do HTML e, em seguida, adicionar a lógica para saber se o Mário encostou**

```
let mario = document.querySelector(".mario");  
let obstaculo = document.querySelector(".obstaculo");
```

```
function marioEnconstou() {  
  const leftObstaculo = parseInt(window.getComputedStyle(obstaculo).left);  
  
  if (leftObstaculo >= 450 && leftObstaculo <= 500) {  
    return true;  
  }  
}
```

O `getComputedStyle` permite que a gente acesse o valor exato da propriedade!

- **Em seguida, vamos validar se o Mário conseguiu pular. Vamos pensar, qual espaço que o Mário ocupa, considerando o `left` e a largura dele?**

Se o Mário pular um valor maior que a altura do obstáculo E cair em uma posição maior que a largura desse mesmo obstáculo, então ele conseguiu pular!

Como ele retorna um texto, precisamos converter para um valor inteiro!

```
function marioNaoPulou() {  
  const bottomMario = parseInt(window.getComputedStyle(mario).bottom);  
  if (bottomMario < 70) {  
    return true;  
  }  
}
```

```
}  
}
```

- **Precisamos colocar tudo dentro de um loop e, para isso, utilizamos o `setInterval`**

```
function game() {  
  loop = setInterval(() => {  
    if (marioEnconstou() && marioNaoPulou()) {  
      console.log("Perdeu");  
    }  
  }, 10);  
}
```

Como o comando, eu a verificação é repetida por 10 milissegundos, o que permite que meu jogo aconteça!

Eu utilizo esse valor para que a minha verificação seja realizada rapidamente, já que um jogo é frenético e se movimenta rapidamente!

Para finalizar o meu jogo eu preciso:

- Que meu loop pare
- Meu personagens fiquem estáticos
- A caixinha para reiniciar o jogo apareça

```
function GameOver() {  
  clearInterval(loop);  
  
  obstaculo.classList.add("gameOver");  
  mario.classList.add("gameOver");  
  
  const div = document.createElement("div");  
  div.classList.add("reiniciar");  
  
  div.innerHTML = `
```

```

    <h2> Game over! </h2>
    <button onclick = "ReiniciarJogo()"> Jogar novamente </
button>

    `;
    //Como a função está no meu Javascript eu não preciso adi
cionar a interpolação?
    reiniciar_jogo.appendChild(div);
}

```

Vamos tentar entender onde cada parte do código está.

- O loop existe no Javascript?
- O obstáculo ou o Mário existem no Javascript?
- E o nome gameOver? Representa o que?

```

.gameOver {
  animation: none;
}

```

Por fim, vamos identificar quais elemento são HTML e quais elementos são CSS?

A partir disso, a gente consegue conferir o que está faltando no nosso código!

- **Agora, precisamos adicionar a lógica para o meu navegador saber quando que o Mário deve pular!**

Além de adicionar a classe que vai fornecer o pulo pro usuário, precisamos, também, resetá-la.

Por isso, colocamos um setTimeout pra ele remover a classe logo depois que ela for adicionada (e 500 milissegundos nada mais é do que o tempo exato da minha animação!)

```

function jump() {
  mario.classList.add("jump");

  setTimeout(() => {

```

```
mario.classList.remove("jump");  
}, 500);  
}
```

```
document.addEventListener("keydown", jump);  
game();
```

Reiniciar o jogo

E agora, para reiniciar o jogo?

1. A gente precisa remover a div
2. Depois, retomar a animação dos dois elementos e....
3. Chamar a função game(), a principal da nossa lógica exatamente por verificar constantemente se teve ou não um vencedor!

Adicione por conta própria esse final e veja o que acontece!